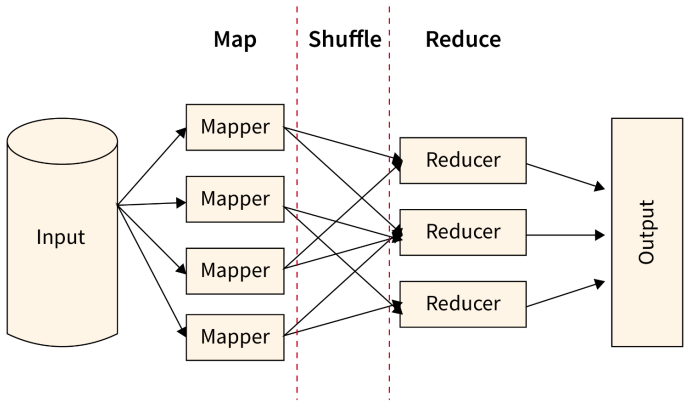
Name:	Krish Sodhi, Marvin Sukumar, Riya Suryawanshi, Anushka Suvarna
UID:	2023300233, 2023300238, 2023300240, 2023300241
Subject:	Distributed Computing
Experiment No.:	8
Project title:	**Travel Itinerary Planner**
Problem Statement:	**Implementation of a basic MapReduce job using Hadoop**
Objectives:	1. To implement a basic MapReduce job using Hadoop to analyze travel data (from the Travel Itinerary Planner) and find the most popular destinations based on user bookings. 2. To understand and apply the MapReduce programming model in Hadoop for large-scale data processing, using destination booking data from a travel itinerary planner system.
Theory:	MapReduce:- 1. MapReduce is a programming model used in Hadoop to process large datasets across multiple computers (nodes) in a parallel and distributed way. 2. Instead of processing data one by one on a single machine, MapReduce splits the work into smaller tasks that can run at the same time on many machines. 

It has two main steps:

1. Map: Processes input data and converts it into intermediate key-value pairs.

2. Reduce**:** Aggregates the intermediate data to produce final results.

Hadoop is an open-source framework that allows us to run MapReduce jobs on a cluster or a single machine using HDFS (Hadoop Distributed File System) for storage.

Distributed Data Processing

- Definition:
  Distributed data processing means splitting a large dataset across multiple computers (nodes) and processing it simultaneously, instead of processing everything on a single machine.

- Why is it needed:
  Modern systems generate huge amounts of data (terabytes or petabytes).
  A single computer cannot handle such data efficiently. Distributing data allows faster processing, reliability, and scalability.

How MapReduce works for our experiment:

1. Input Data:
   The dataset consists of user booking information from the Travel Itinerary Planner. Each record may include a user ID, destination, travel date, and other details.

2. Map Phase:

   - Each Mapper reads the input records line by line.
   - For each record, it extracts the destination and emits a key-value pair: (destination, 1)
   - This means each time a destination appears in the dataset, it is counted once.

3. Shuffle and Sort:

   - Hadoop automatically groups all values for the same key (destination) together.

   - For example, all counts of “Jaipur” will be collected together.

	4. Reduce Phase: ○ The Reducer takes each destination and the list of counts and user feedback ratings ○ It sums up the counts to get the average user rating for each location 5. Output: ○ The final output shows all destinations with their average rating from users ○ We can easily identify the most popular destinations based on these counts. Advantages of Using Hadoop MapReduce: 1. Handles large datasets efficiently. 2. Automatically distributes tasks across multiple nodes (or cores on a single machine). 3. Fault-tolerant and scalable. 4. Demonstrates the classic "many Mappers, one Reducer" pattern.
Code:-	**Docker-compose.yml :-** version: '3.8' services: hadoop: image: harisekhon/hadoop:latest container_name: hadoop-single-node ports: - "9870:9870" - "9864:9864" - "8088:8088" - "19888:19888" volumes: - ./hadoop_ingest:/tmp/ingest - ./mapreduce:/tmp/mapreduce environment: - HADOOP_HEAPSIZE=512

coordinator.py :-

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import requests
from concurrent.futures import ThreadPoolExecutor, as_completed
import sqlite3
import os
import json
from datetime import datetime
import subprocess

app = Flask("coordinator")

CORS(app, resources={r"/*": {"origins": "http://localhost:8080"}})
DB = "travel.db"
INGEST_FOLDER ="hadoop_ingest"

NODES = {
   "restaurants": "http://localhost:5001/search",
   "events": "http://localhost:5002/search",
   "transport": "http://localhost:5003/search",
   "hotels": "http://localhost:5004/search"
}

def call_node(url, params):
   try:
      r = requests.get(url, params=params, timeout=5)
      r.raise_for_status()
      return r.json()
   except Exception as e:
      return {"source": url, "error": str(e), "results": []}

def aggregate_results(node_responses, max_items=3):
   lists = []
   for nr in node_responses:
      source = nr.get("source", "unknown")
      results = nr.get("results", [])
      tagged_results = []
      for item in results[:max_items]:

         tagged_results.append({**item, "category": source})
      lists.append(tagged_results)
   itinerary = []
   for i in range(max_items):
      for lst in lists:
         if i < len(lst):
            itinerary.append(lst[i])
   return itinerary
```

```
@app.route("/plan")
def plan():
    city = request.args.get("city", "")
    if not city:
        return jsonify({"error": "city parameter required"}), 400
    max_per_node = int(request.args.get("max_per_node", 3))
    params = {"city": city, "limit": max_per_node}
    responses = []
    with ThreadPoolExecutor(max_workers=len(NODES)) as ex:
        futures = {ex.submit(call_node, url, params): name for name, url in NODES.items()}
        for future in as_completed(futures):
            name = futures[future]
            result = future.result()
            result["category"] = name
            responses.append(result)

    itinerary = aggregate_results(responses, max_items=max_per_node)
    print(itinerary)
    return jsonify({"city": city, "itinerary": itinerary, "node_responses": responses})

@app.route("/feedback", methods=["POST"])
def feedback():
    data = request.get_json()
    required = ["user_id", "location", "itinerary", "rating"]
    if not data or not all(k in data for k in required):
        return jsonify({"error": f"Required fields: {required}"}), 400

    user_id = data["user_id"]
    location = data["location"]
    itinerary = data["itinerary"]
    rating = float(data["rating"])
    comment = data.get("comment", "")
    timestamp = datetime.utcnow().isoformat()

    conn = sqlite3.connect(DB)
    c = conn.cursor()
    c.execute("INSERT INTO feedback (user_id, location, itinerary, rating, comment, timestamp)
VALUES (?, ?, ?, ?, ?, ?)",
              (user_id, location, json.dumps(itinerary), rating, comment, timestamp))
    conn.commit()
    conn.close()

    filename = f"feedback_{datetime.utcnow().strftime('%Y%m%dT%H%M%S%f')}.json"
    filepath = os.path.join(INGEST_FOLDER, filename)
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump({
            "user_id": user_id,
            "location": location,
            "itinerary": itinerary,
```

```
            "rating": rating,
            "comment": comment,
            "timestamp": timestamp
        }, f, ensure_ascii=False)

    return jsonify({"status": "ok", "stored": True, "ingest_file": filepath})



if __name__ == "__main__":
    app.run(port=5000, debug=True)
```

Feedback_descriptive.py :-

```
import sys
import json

def emit(city, rating):

        rating = float(rating)
        print "%s\t%s" % (city, rating)


def mapper():
    buffer = ""
    for line in sys.stdin:
        line = line.strip()
        if not line:
            continue
        buffer += line

         data = json.loads(buffer)
         buffer = ""  # clear buffer if parsed successfully

         city = data.get('location', 'unknown')
         emit(city, data.get('rating', 0))

         for item in data.get('itinerary', []):
               item_city = item.get('city', city)
               emit(item_city, item.get('rating', 0))


def reducer():
    current_city = None
    total_rating = 0
    count = 0

    for line in sys.stdin:
        line = line.strip()
        if not line:
```

```
            continue

        city, rating = line.split("\t")
        rating = float(rating)

        if current_city == city:
            total_rating += rating
            count += 1
        else:
            if current_city:
                avg_rating = total_rating / count
                print "City: %s" % current_city
                print "  Average Rating: %.2f" % avg_rating
                print "  Feedback Count: %d" % count
                print "---"
            current_city = city
            total_rating = rating
            count = 1

    if current_city:
        avg_rating = total_rating / count
        print "City: %s" % current_city
        print "  Average Rating: %.2f" % avg_rating
        print "  Feedback Count: %d" % count

if __name__ == "__main__":
    if len(sys.argv) > 1 and sys.argv[1] == "reduce":
        reducer()
    else:
        mapper()
```

Terminal Commands:-

```
docker-compose up -d

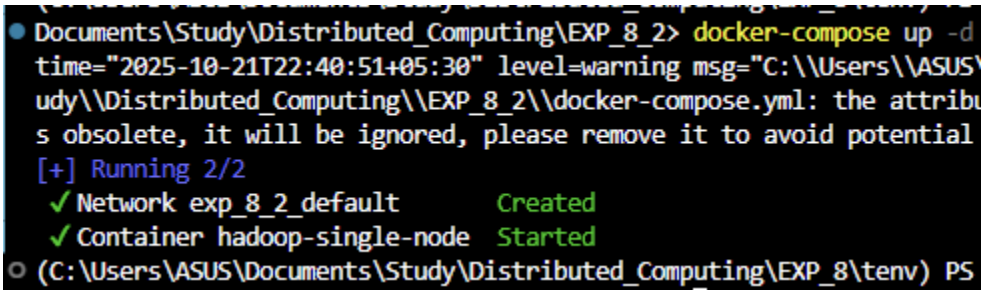
docker exec -it hadoop-single-node
hdfs dfs -mkdir -p /user/travel/feedback
hdfs dfs -put -f /tmp/ingest/feedback_*.json /user/travel/feedback/

hadoop jar /hadoop-2.8.2/share/hadoop/tools/lib/hadoop-streaming-2.8.2.jar \
  -files /tmp/ingest/feedback_descriptive.py \
  -mapper "python feedback_descriptive.py" \
  -reducer "python feedback_descriptive.py reduce" \
  -input /user/travel/feedback/*.json \
  -output /user/travel/output

hdfs dfs -cat /user/travel/output/part-00000
```

Output:-	**Command:-**

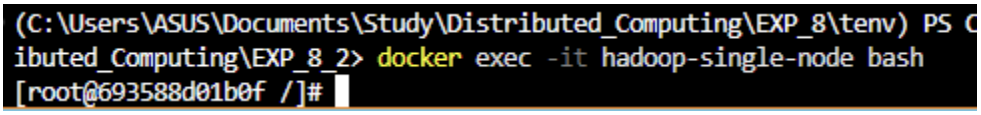
➢ **docker-compose up -d :-**



Container created:- Yes

Command:-

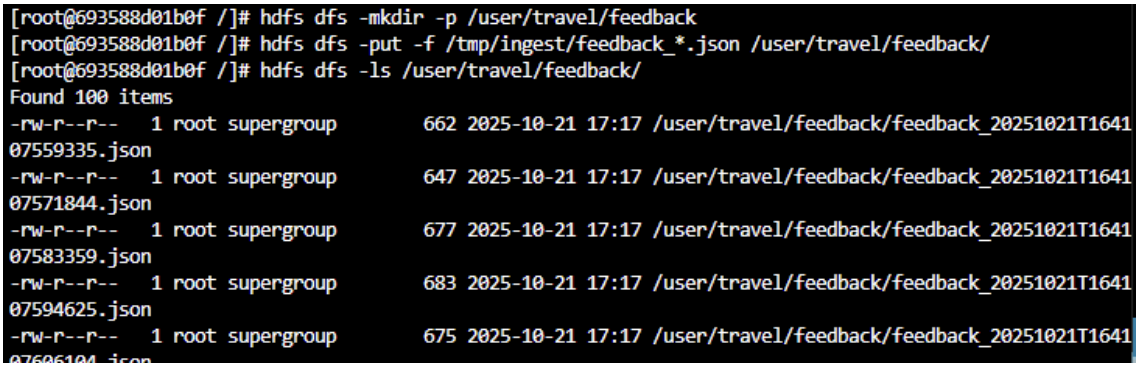
➢ **docker exec -it hadoop-single-node**



Entered Container:- Yes

Commands:-

➢ **hdfs dfs -mkdir -p /user/travel/feedback**
➢ **hdfs dfs -put -f /tmp/ingest/feedback_*.json /user/travel/feedback/**
➢ **hdfs dfs -ls /user/travel/feedback/**



Feedback Files copied to container:- Yes

Command:-

➢ **hadoop jar /hadoop-2.8.2/share/hadoop/tools/lib/hadoop-streaming-2.8.2.jar **
**-files /tmp/ingest/feedback_descriptive.py **
**-mapper "python feedback_descriptive.py" **
**-reducer "python feedback_descriptive.py reduce" **
**-input /user/travel/feedback/*.json **
-output /user/travel/output

```
[root@5216e9b6752d /]# hdfs dfs -rm -r -f /user/travel/output && hadoop jar /hadoop-2.8.2/share/hadoop/
tools/lib/hadoop-streaming-2.8.2.jar   -files /tmp/ingest/feedback_descriptive.py   -mapper "python fee
dback_descriptive.py"   -reducer "python feedback_descriptive.py reduce"   -input /user/travel/feedback
/*.json   -output /user/travel/output && hdfs dfs -cat /user/travel/output/part-00000
Deleted /user/travel/output
packageJobJar: [/tmp/hadoop-unjar414787514513399051/] [] /tmp/streamjob3584976803257007133.jar tmpDir=n
ull
25/10/21 16:41:46 INFO client.RMProxy: Connecting to ResourceManager at /0.0
25/10/21 16:41:46 INFO client.RMProxy: Connecting to ResourceManager at /0.0
25/10/21 16:41:47 INFO mapred.FileInputFormat: Total input files to process
25/10/21 16:41:47 INFO mapreduce.JobSubmitter: number of splits:100
25/10/21 16:41:47 INFO mapreduce.JobSubmitter: Submitting tokens for job: jo
25/10/21 16:41:47 INFO impl.YarnClientImpl: Submitted application applicatio
25/10/21 16:41:47 INFO mapreduce.Job: The url to track the job: http://5216e
ion_1761053719847_0013/
25/10/21 16:41:47 INFO mapreduce.Job: Running job: job_1761053719847_0013
25/10/21 16:41:53 INFO mapreduce.Job: Job job_1761053719847_0013 running in
```

Hadoop Streaming Job started:- Yes

```
25/10/21 16:42:50 INFO mapreduce.Job:  map 74% reduce 23%
25/10/21 16:42:54 INFO mapreduce.Job:  map 79% reduce 23%
25/10/21 16:42:56 INFO mapreduce.Job:  map 79% reduce 26%
25/10/21 16:42:58 INFO mapreduce.Job:  map 84% reduce 26%
25/10/21 16:43:02 INFO mapreduce.Job:  map 89% reduce 28%
25/10/21 16:43:06 INFO mapreduce.Job:  map 94% reduce 28%
25/10/21 16:43:07 INFO mapreduce.Job:  map 94% reduce 31%
25/10/21 16:43:10 INFO mapreduce.Job:  map 99% reduce 31%
25/10/21 16:43:12 INFO mapreduce.Job:  map 100% reduce 31%
25/10/21 16:43:13 INFO mapreduce.Job:  map 100% reduce 100%
25/10/21 16:43:13 INFO mapreduce.Job: Job job_1761053719847_0013 completed successfully
25/10/21 16:43:13 INFO mapreduce.Job: Counters: 50
```

Hadoop Streaming Job completed successfully:- Yes

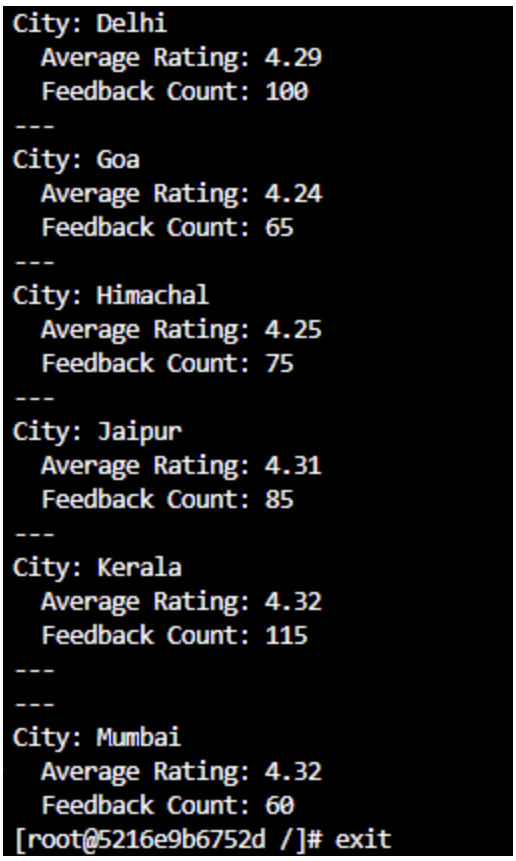
```
        File System Counters
                FILE: Number of bytes read=6361
                FILE: Number of bytes written=14302023
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=79762
                HDFS: Number of bytes written=385
                HDFS: Number of read operations=303
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Killed map tasks=1
                Launched map tasks=100
                Launched reduce tasks=1
                Data-local map tasks=100
                Total time spent by all maps in occupied slots (ms)=294071
                Total time spent by all reduces in occupied slots (ms)=62143
                Total time spent by all map tasks (ms)=294071
                Total time spent by all reduce tasks (ms)=62143
                Total vcore-milliseconds taken by all map tasks=294071
                Total vcore-milliseconds taken by all reduce tasks=62143
                Total megabyte-milliseconds taken by all map tasks=301128704
                Total megabyte-milliseconds taken by all reduce tasks=63634432
```

```
Map-Reduce Framework
        Map input records=100
        Map output records=500
        Map output bytes=5355
        Map output materialized bytes=6955
        Input split bytes=13300
        Combine input records=0
        Combine output records=0
        Reduce input groups=6
        Reduce shuffle bytes=6955
        Reduce input records=500
        Reduce output records=23
        Spilled Records=1000
        Shuffled Maps =100
        Failed Shuffles=0
        Merged Map outputs=100
        GC time elapsed (ms)=9980
        CPU time spent (ms)=48400
        Physical memory (bytes) snapshot=28166492160
        Virtual memory (bytes) snapshot=205135605760
        Total committed heap usage (bytes)=20281556992
Shuffle Errors
        BAD_ID=0
        CONNECTION=0
        IO_ERROR=0
        WRONG_LENGTH=0
        WRONG_MAP=0
        WRONG_REDUCE=0
```

```
        File Input Format Counters
                Bytes Read=66462
        File Output Format Counters
                Bytes Written=385
25/10/21 16:43:13 INFO streaming.StreamJob: Output directory: /user/travel/output
```

Command:-

- **hdfs dfs -cat /user/travel/output/part-00000**

	 **Output Displayed Correctly:-Yes**
Conclusion:	By implementing this MapReduce job, we can efficiently process the travel booking data and determine the most popular destinations. This experiment demonstrates both the MapReduce programming model and Hadoop's ability to handle real-world datasets.